Faculty of Engineering & Technology

Electrical & Computer Engineering Department

Artificial Intelligence - ENCS3340

Project 1

Arabic Sentiment Analysis

Prepared by:	Nancy Samhan	1230926.
	Lina Hamad	1231412.
Instructor:	Dr. Yazan Abu Farha.	
Section:	1.	
Date:	January 23, 2026	

Table of Contents

Table of Contents I

Table of Figures II

1.Introduction 1

2. Dataset Description 2

3. Exploratory Data Analysis (EDA) 2

4. Text Preprocessing 3

5. Feature Extraction 3

6. Handcrafted Feature Engineering 4

7. Model Training and Hyperparameter Tuning 7

- **7.1 Decision Tree Classifier** 8
- **7.2 Random Forest Classifier** 9
- **7.3 Naive Bayes Classifier** 10
- **7.4 Neural Network Model** 11

8. Results Comparison and Discussion 12

- **8.1 Confusion Matrix Analysis** 12
- **8.2 Quantitative Performance Comparison** 13
- **8.3 Interpretation of Accuracy and F1-Score** 15
- **8.4 Discussion** 16

Table of Figures

Figure 1:Output of Decision Tree classifier. 8
Figure 2: Output of Random Forest classifier. 9
Figure 3: Output of Naive Bayes classifier. 10
Figure 4: Output of Neural Network classifier. 11
Figure 5: Confusion matrices of all evaluated models. 13
Figure 6:The winner. 13
Figure 7: Model performance comparison in terms of Accuracy and weighted F1-score. 14

1. Introduction

Machine learning is a core field of artificial intelligence that focuses on developing algorithms capable of learning patterns and making predictions from data without being explicitly programmed. By automatically identifying relationships within large and complex datasets, machine learning techniques have become essential in solving a wide range of real-world problems, including text classification, image recognition, recommendation systems, and speech processing.

This project aims to develop an Arabic sentiment analysis system for classifying social media tweets into three sentiment categories: **Positive (POS)**, **Negative (NEG)**, and **Objective/Neutral (OBJ)**. The dataset used in this study consists of Arabic tweets annotated with these sentiment labels and exhibits a noticeable class imbalance, which can negatively affect classification performance if not properly handled.

Sentiment analysis is a fundamental task in natural language processing (NLP) that focuses on automatically identifying the emotional polarity expressed in textual data. With the rapid growth of social media platforms, sentiment analysis has become an important tool for understanding public opinion, user attitudes, and social trends. However, performing sentiment analysis on Arabic text remains challenging due to the complex morphology of the language, the wide variety of dialects, informal writing styles, spelling inconsistencies, and the presence of noisy user-generated content such as emojis, hashtags, and abbreviations.

To address these challenges, a **hybrid feature extraction approach** was adopted. First, **contextual embeddings were generated using AraBERT**, a pre-trained transformer-based language model specifically designed for Arabic. AraBERT embeddings capture deep semantic and contextual information from the text, allowing the model to better understand meaning beyond surface-level word representations. Second, a set of **handcrafted sentiment-related features** was extracted to explicitly capture sentiment cues such as the presence of positive and negative words, sentiment-bearing emojis, negation terms, punctuation marks, and basic text statistics. Combining these two types of features enables the model to leverage both high-level semantic representations and explicit sentiment indicators.

Several machine learning classifiers were trained using the extracted features, including **Decision Tree**, **Random Forest**, and **Naive Bayes**, in addition to an optional **Neural Network** model. To ensure robust performance, hyper parameter tuning was performed using GridSearch with cross-validation, and model evaluation was conducted using standard classification metrics such as Accuracy, Precision, Recall, and F1-score. The experimental results were then compared to identify the most effective model for Arabic sentiment classification.

2. Dataset Description

The dataset used in this project consists of Arabic tweets annotated with sentiment labels representing Positive, Negative, and Objective/Neutral emotions. Each data sample includes the tweet text and its corresponding sentiment label.

The dataset was stored as a UTF-8 encoded, tab-separated text file to ensure proper processing of Arabic characters. During data loading, all sentiment labels were standardized by converting them to uppercase, and the **NEUTRAL** label was merged with the **OBJ** class to maintain consistency across sentiment categories. As part of the data preparation process, malformed records, empty tweets, and non-informative samples were removed. This cleaning process reduced noise and improved the overall quality of the dataset used for training and evaluation.

3. Exploratory Data Analysis (EDA)

An exploratory analysis of the dataset showed a clear **class imbalance**, with the **Objective/Neutral (OBJ)** class being significantly more frequent than the Positive and Negative classes. For instance, the dataset contained approximately **OBJ: 7,523 samples**, **POS: 799 samples**, and **NEG: 1,684 samples** *(illustrative values later replaced by actual counts obtained from the code execution).*

This imbalance is common in social media data, as many tweets are factual or informative rather than explicitly expressing positive or negative sentiment. Such imbalance can bias classifiers toward the majority class and reduce performance on minority classes. To handle this issue, imbalance-handling techniques such as **class weighting** and **SMOTE** were applied during model training.

4. Text Preprocessing

To prepare the Arabic text for feature extraction, a comprehensive preprocessing pipeline was applied. The preprocessing steps included removing URLs, user mentions, and unnecessary symbols, normalizing Arabic characters to reduce orthographic variations, and removing diacritics. Repeated characters were reduced, and non-Arabic content such as English letters and numbers was eliminated.

As a result of these cleaning operations, the dataset size was reduced. This reduction occurred because some tweets were empty, malformed, or became non-informative after preprocessing (e.g., tweets containing only links, symbols, or non-Arabic text). Removing these samples improved data quality and ensured that only meaningful textual content was used for feature extraction and model training.

5. Feature Extraction

Two complementary types of features were extracted from the tweets. First, contextual embeddings were generated using **AraBERT**, a pre-trained language model based on the **Transformer architecture**. Transformers rely on a self-attention mechanism that enables the model to analyze all words in a sentence simultaneously and capture long-range dependencies between them. Unlike traditional models that process text sequentially, Transformers generate context-aware representations, allowing each word to be interpreted based on its surrounding words.

AraBERT is specifically trained on large-scale Arabic corpora, enabling it to effectively model the linguistic complexity, morphology, and contextual variations of the Arabic language. In this work, the embedding of the special **[CLS] token** was used as a fixed-length representation of each tweet, producing a dense vector that captures the overall semantic meaning of the entire sentence.

Second, a set of handcrafted features was extracted to capture explicit sentiment indicators such as the presence of positive and negative words, sentiment-related emojis, negation terms, punctuation marks, and word-level statistics. Combining contextual embeddings with handcrafted features allowed the models to leverage both deep semantic understanding and surface-level sentiment cues, resulting in a richer and more discriminative feature representation.

6. Handcrafted Feature Engineering

To complement AraBERT contextual embeddings, we extracted a compact set of **12 handcrafted features** designed to capture explicit sentiment signals that frequently appear in social media Arabic tweets (e.g., emojis, sentiment lexicon matches, negation cues, punctuation intensity, and simple length statistics). This hybrid design allows the model to benefit from both deep semantic representations (AraBERT) and surface-level cues that are especially informative in informal user-generated text.

1) Emoji-based features

1. **pos_emoji**: Counts the number of emojis in the original tweet that belong to a predefined **positive emoji set** (e.g., ☺ ❤ 👍 🎉).
   Motivation: Emojis often directly encode affect and can signal positivity even when text is short or ambiguous.
2. **neg_emoji**: Counts the number of emojis that belong to a predefined **negative emoji set** (e.g., 😒 💔 😟 👎).
   Motivation: Negative emojis are strong indicators of sadness, anger, disappointment, or frustration.

Limitation: Emoji inventories are inevitably incomplete; some commonly used emojis (e.g., sarcasm-related emojis) may not be covered, which can reduce recall for certain sentiment expressions.

2) Lexicon-based sentiment word counts

3. **pos_word**: Counts the number of tokens in the processed tweet that match a curated list of **positive Arabic words** (e.g., "ممتاز", "رائع", "جميل").
   Motivation: Explicit positive words provide direct sentiment evidence, especially in short tweets.
4. **neg_word**: Counts the number of tokens that match a curated list of **negative Arabic words** (e.g., "مقرف", "حزن", "سيء").
   Motivation: Negative keywords strongly correlate with negative sentiment polarity.

Limitations:

- Dialectal and slang variants may not be included in the lexicon, reducing coverage in real-world tweets.
- Simple exact matching may miss morphological variants (e.g., different inflections, clitics, or spelling variations).

3) Negation feature

5. **negation**: Counts occurrences of Arabic negation terms (e.g., “ ,”ليس“ ,”لن“ ,”لم“ ,”لا“ ,”ما“ ,”مش“”).
   Motivation: Negation can flip sentiment polarity (e.g., “مش حلو” vs. “حلو”), so detecting negation provides an important cue.

Limitation: This feature counts negations but does not explicitly model **negation scope** (i.e., which words are negated). As a result, phrases like “مش ممتاز” may still contain positive lexical matches without capturing the polarity reversal.

4) Punctuation-based intensity features

6. **exclaim**: Counts the number of exclamation marks “!” in the original tweet.
   Motivation: Exclamation marks often indicate emotional intensity (enthusiasm, anger, excitement).
7. **question**: Counts occurrences of “?” and the Arabic question mark “؟”.
   Motivation: Questions can express uncertainty, complaint, or rhetorical sentiment (e.g., frustration or disbelief).

Limitation: Punctuation does not uniquely map to polarity; it mainly captures intensity and must be interpreted jointly with other signals.

5) Basic length/statistical features

8. **char_count**: Number of characters in the original tweet.

   Motivation: Longer tweets may contain more context; extremely short tweets may rely more on emojis or single sentiment words.

9. **word_count**: Number of words after preprocessing and tokenization.

   Motivation: Provides a coarse indicator of content richness.

10. **char_repeat**: Counts repeated-character patterns (e.g., “لاااا“ ,”حلووو”).

    Motivation: Character elongation is a common social media convention that indicates emphasis and emotional intensity.

11. **avg_word_len**: Average length of tokens in the processed tweet.

    Motivation: Captures stylistic differences (informal vs. formal writing), abbreviations, and short expressive tokens.

Limitation: Some of these statistics may be weak predictors individually; their value is typically additive when combined with stronger semantic features.

6) Aggregate sentiment score

12. **sentiment_score**: A simple polarity proxy computed as:

    $$sentiment_score = (pos_word+pos_emoji) - (neg_word+neg_emoji)$$

Motivation: Provides a lightweight summary of explicit positive vs. negative cues, which can be especially useful under class imbalance or noisy text conditions.

Limitation: This score treats all cues equally (uniform weights) and does not incorporate negation scope or intensity modifiers.

Why Transformer-Based Embeddings Were Preferred?

Transformer-based embeddings such as **AraBERT** were preferred over traditional approaches like **TF-IDF** and **Word2Vec** due to their ability to capture contextual and semantic information more effectively. TF-IDF represents text as sparse vectors based solely on word frequency and does not consider word order or context, which limits its ability to distinguish between different meanings of the same word in varying contexts.

Word2Vec improves upon TF-IDF by learning dense word embeddings; however, it assigns a single fixed vector to each word regardless of context. This limitation is particularly problematic in Arabic, where word meaning can change significantly depending on surrounding words, morphology, and syntactic structure.

In contrast, Transformer-based models generate **context-aware embeddings**, where the representation of a word dynamically changes according to its context. AraBERT leverages self-attention mechanisms to model long-range dependencies and complex linguistic patterns in Arabic text. As a result, AraBERT embeddings provide a richer and more expressive representation, which leads to improved performance, especially in noisy and imbalanced social media datasets.

7. Model Training and Hyperparameter Tuning

To evaluate the effectiveness of different learning approaches for Arabic sentiment classification, multiple models were trained using the combined feature set consisting of **AraBERT embeddings (768 features)** and **12 handcrafted sentiment features**, resulting in a total of **780 features** per tweet. Due to the significant class imbalance in the dataset, **SMOTE** was applied to the training set only, producing a balanced distribution of **4,365 samples per class**. Feature scaling was then applied using StandardScaler to ensure numerical stability, particularly for models sensitive to feature magnitude.

- **Dataset Splitting and Evaluation Protocol**

The dataset was partitioned into three disjoint subsets to ensure reliable training and evaluation. Specifically, 60% of the data was used for training, 20% for validation, and 20% for testing. The training set was used to fit the models, the validation set was employed for hyperparameter tuning

and model selection, and the test set was held out for final performance evaluation. All reported results were obtained using the test set to ensure an unbiased assessment.

Hyperparameter tuning for all classical machine learning models was performed using **GridSearch with 3-fold cross-validation**, where the **F1-score** was used as the optimization metric to account for class imbalance.

➢ 7.1 Decision Tree Classifier

The **Decision Tree** classifier was used as an interpretable baseline model. Decision Trees operate by recursively splitting the feature space to maximize class separation, allowing them to capture non-linear relationships between features.

After hyperparameter tuning, the optimal configuration included a maximum depth of **30**, a minimum of **5 samples per split**, and **2 samples per leaf**. Despite these optimizations, the Decision Tree achieved a relatively low **test accuracy of 56.2%** and a **weighted F1-score of 0.595**. The confusion matrix showed that the model struggled to correctly classify minority classes, particularly the **Positive (POS)** class, which suffered from low precision and recall. This behavior indicates that a single tree was insufficient to generalize well over the high-dimensional feature space.

```
================================================================================
Decision Tree - Test Accuracy: 0.5619
================================================================================
              precision    recall  f1-score   support

         NEG       0.24      0.37      0.29       337
         OBJ       0.79      0.63      0.70      1505
         POS       0.16      0.29      0.21       160

    accuracy                           0.56      2002
   macro avg       0.40      0.43      0.40      2002
weighted avg       0.65      0.56      0.59      2002
```

Figure 1:Output of Decision Tree classifier.

7.2 Random Forest Classifier

The **Random Forest** classifier was employed as an ensemble-based model to overcome the limitations of a single decision tree. Random Forest constructs multiple decision trees using random subsets of data and features and aggregates their predictions, which significantly reduces overfitting and improves generalization.

The best-performing Random Forest model used **300 trees** with a maximum depth of **30** and a minimum of **5 samples per split**. This model achieved the **highest performance among all evaluated classifiers**, with a **test accuracy of 75.2%** and a **weighted F1-score of 0.738**. The model demonstrated strong performance on the majority **OBJ** class while also improving classification of minority classes compared to the Decision Tree. The ensemble nature of Random Forest allowed it to better exploit the rich semantic information provided by AraBERT embeddings.

```
==============================================================================
Random Forest - Test Accuracy: 0.7517
==============================================================================
              precision    recall  f1-score   support

         NEG       0.49      0.43      0.46       337
         OBJ       0.82      0.88      0.85      1505
         POS       0.43      0.23      0.30       160

    accuracy                           0.75      2002
   macro avg       0.58      0.51      0.54      2002
weighted avg       0.73      0.75      0.74      2002
```

Figure 2: Output of Random Forest classifier.

➢ 7.3 Naive Bayes Classifier

The **Naive Bayes** classifier was included as a probabilistic baseline due to its computational efficiency and simplicity. Gaussian Naive Bayes was used to accommodate the continuous-valued feature space generated by AraBERT embeddings.

After tuning the smoothing parameter, the model achieved a **test accuracy of 58.5%** and a **weighted F1-score of 0.62**. While Naive Bayes showed relatively high recall for the **Negative (NEG)** class, its overall performance was limited. This can be attributed to the strong independence assumptions made by Naive Bayes, which are not well-suited for dense contextual embeddings where features are highly correlated.

```
==============================================================================
Naive Bayes - Test Accuracy: 0.5849
==============================================================================
              precision    recall  f1-score   support

         NEG       0.35      0.67      0.46       337
         OBJ       0.85      0.58      0.69      1505
         POS       0.20      0.41      0.27       160

    accuracy                           0.58      2002
   macro avg       0.47      0.56      0.48      2002
weighted avg       0.72      0.58      0.62      2002
```

Figure 3: Output of Naive Bayes classifier.

➢ 7.4 Neural Network Model

A feedforward **Neural Network** was trained to further explore the effectiveness of deep learning on the extracted features. The network consisted of two hidden layers (128 and 64 neurons) with ReLU activation functions and dropout regularization to reduce overfitting. Early stopping was applied to prevent unnecessary training once validation performance stopped improving.

The Neural Network converged after **6 epochs** and achieved a **test accuracy of 68.1%** with a **weighted F1-score of 0.701**. While the model performed better than Naive Bayes and Decision Tree, it did not surpass the Random Forest classifier. This suggests that ensemble methods were more effective than a shallow neural network in leveraging the combined feature representation under the given training constraints.

```
================================================================================
Neural Network - Test Accuracy: 0.6808
================================================================================
              precision    recall  f1-score   support

         NEG       0.42      0.60      0.49       337
         OBJ       0.86      0.72      0.78      1505
         POS       0.31      0.47      0.37       160

    accuracy                           0.68      2002
   macro avg       0.53      0.60      0.55      2002
weighted avg       0.74      0.68      0.70      2002
```

Figure 4: Output of Neural Network classifier.

8. Results Comparison and Discussion

This section presents a comprehensive comparison of the evaluated models and discusses their performance based on quantitative metrics and visual analysis. The models were assessed using the held-out test set to ensure an unbiased evaluation.

➢ 8.1 Confusion Matrix Analysis

Figure 1 illustrates the confusion matrices of the Decision Tree, Random Forest, Naive Bayes, and Neural Network models. These matrices provide detailed insight into the classification behavior of each model across the three sentiment classes: Positive (POS), Negative (NEG), and Objective (OBJ).

The Decision Tree model demonstrates substantial misclassification across sentiment categories, particularly for the minority POS class, indicating limited generalization capability. Similarly, the Naive Bayes classifier shows moderate performance but struggles to correctly distinguish between sentiment classes due to its strong feature independence assumptions.

In contrast, the Random Forest model exhibits a significantly improved classification pattern, achieving high accuracy for the dominant OBJ class while also enhancing performance on the NEG class. The Neural Network shows more balanced recall across classes compared to Decision Tree and Naive Bayes; however, it still encounters difficulties in accurately classifying POS samples, largely due to the severe class imbalance present in the dataset.

Figure 5: Confusion matrices of all evaluated models.

➢ 8.2 Quantitative Performance Comparison

The overall performance comparison of the models is presented in **Figure 2**, which compares Accuracy and weighted F1-score for each classifier.

As shown in Figure 2, the Random Forest classifier outperforms all other models, achieving the highest test accuracy (**75.2%**) and weighted F1-score (**0.738**). This indicates that ensemble-based approaches are particularly effective when applied to high-dimensional contextual embeddings such as AraBERT. The Neural Network achieves competitive results with a test accuracy of **68.1%**, benefiting from non-linear feature learning but remaining sensitive to class imbalance.

```
================================================================================
MODEL COMPARISON
================================================================================
         Model  Test_Accuracy  Test_F1
 Decision Tree       0.561938 0.594913
 Random Forest       0.751748 0.737976
   Naive Bayes       0.584915 0.620087
Neural Network       0.680819 0.700572

🏆 Best Model: Random Forest
   Accuracy: 0.7517
   F1-Score: 0.7380
```

Figure 6:The winner.

The Decision Tree and Naive Bayes models achieve lower overall performance, highlighting their limitations in modeling complex semantic relationships present in contextualized word representations.

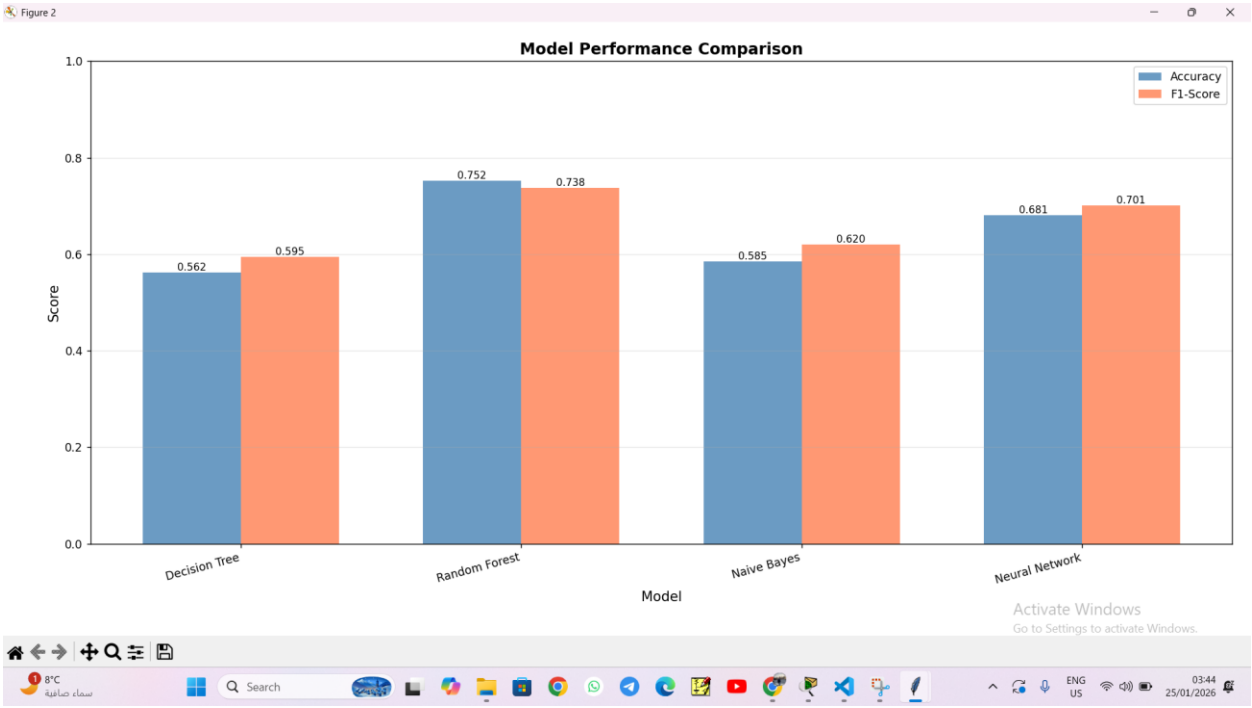

Figure 7: Model performance comparison in terms of Accuracy and weighted F1-score.

➢ 8.3 Interpretation of Accuracy and F1-Score

Accuracy and F1-score are commonly used evaluation metrics for classification tasks; however, they capture different aspects of model performance. **Accuracy** measures the overall proportion of correctly classified instances among all samples. A **high accuracy** indicates that the model predicts the majority of samples correctly, while a **low accuracy** suggests frequent misclassifications. Although accuracy is intuitive, it can be misleading in datasets with **class imbalance**, where one or more classes dominate the dataset. In such cases, a model may achieve high accuracy by favoring the majority class while performing poorly on minority classes.

The **F1-score** provides a more balanced evaluation by combining **precision** (the proportion of correct positive predictions) and **recall** (the proportion of actual positives that are correctly identified) through their harmonic mean. A **high F1-score** indicates that the model achieves both high precision and high recall, meaning it not only makes correct predictions but also successfully captures most samples of each class.

In our case, the **F1-score was relatively high** because the model was explicitly optimized using **weighted F1-score** during training and validation, which assigns higher importance to minority classes. Additionally, the application of **SMOTE oversampling** reduced class imbalance in the training set, allowing the classifiers to better learn decision boundaries for underrepresented sentiment classes. Finally, the hybrid feature representation—combining **AraBERT contextual embeddings** with **handcrafted sentiment features**—enabled the model to capture both deep semantic information and explicit sentiment cues (such as emojis, sentiment words, and negation), leading to improved recall without sacrificing precision. As a result, the F1-score increased, reflecting a more balanced and robust classification performance across all sentiment classes.

➢ 8.4 Discussion

The experimental results demonstrate that model performance is strongly influenced by both **feature representation** and **class distribution**. The dominance of the **OBJ (neutral)** class resulted in consistently higher recall for neutral samples across all evaluated models, while the **POS (positive)** class proved to be the most challenging to classify due to its relatively limited number of instances and lexical variability in informal Arabic tweets.

The superior performance of the **Random Forest** model can be attributed to its **ensemble-based learning strategy**, which aggregates multiple decision trees to reduce overfitting and effectively capture complex, non-linear interactions among features. This capability is particularly beneficial when combining heterogeneous feature types, such as dense AraBERT embeddings and sparse handcrafted sentiment indicators.

Furthermore, the integration of **AraBERT contextual embeddings**, **handcrafted sentiment features**, and **imbalance-aware training techniques** (specifically SMOTE and weighted F1-score optimization) played a crucial role in improving overall classification robustness. These components enabled the models to better recognize minority sentiment classes while maintaining strong precision for the dominant class.

Overall, the findings emphasize the importance of **hybrid feature representations** and **ensemble learning methods** when addressing Arabic sentiment analysis tasks involving **noisy, informal, and imbalanced social media data**.

Please see the README file for a detailed explanation of the project structure, methodology, and implementation details.